Universidad San Carlos de Guatemala
Facultad de ingeniería.
Ingeniería en ciencias y sistemas

Proyecto Fase 2

Backend y Exposición de Servicios para la Base de Datos "Comercial La Estrella"

PONDERACIÓN: 35.72 pts

Tiempo estimado: 40 hrs/min

Índice

1. MARCO FORMATIVO 3
1.1. Valor 3
1.2. Competencia(s) 3
1.3. Habilidad(es) blandas a formar 3
2. Resultado del Aprendizaje 4
2.1. Objetivo SMART 4
3. Enunciado de la Práctica 4
3.1 Descripción del problema a resolver 4
3.2 Alcance de la práctica 5
3.3 Guía Recomendada para la Importación de Datos 5
3.4 Requerimientos técnicos 6
4. Entregables 6
5. Material de apoyo 7
6. Recursos y herramientas a utilizar 7
7. Cronograma 7
8. Rúbrica de Calificación 8
8.1 Requisitos para optar a la calificación 8
8.2 Resumen de Puntuaciones 9
Detalle de la Calificación 10
Comentarios Generales 11

1. MARCO FORMATIVO

1.1. Valor

Nombre del valor	¿Cómo se aplica en tu laboratorio?
Integridad académica y responsabilidad profesional	El estudiante debe configurar su propia infraestructura en Docker, desarrollar su API REST y programar sus objetos de base de datos (vistas y triggers) en SQL Server de forma estrictamente individual. No se permite copiar código, clonar soluciones de compañeros o delegar el trabajo, ya que el desarrollo del criterio lógico es esencial para su formación profesional.

1.2. Competencia(s)

Con la elaboración de este proyecto usted adquirirá la siguiente competencia: Implementa, protege y expone bases de datos relacionales mediante la creación de un entorno dockerizado y el desarrollo de un API REST, delegando reglas de negocio complejas y consultas analíticas al motor de base de datos SQL Server mediante el uso de Triggers y Vistas.

1.3. Habilidad(es) blandas a formar

El proyecto le permitirá desarrollar las siguientes habilidades:

- **Pensamiento analítico y resolución de problemas:** Capacidad de aislar responsabilidades de software, decidiendo qué lógica debe pertenecer a la capa de la base de datos (validaciones de inventario y estados) y qué lógica pertenece a la capa de aplicación.

- **Gestión de calidad y organización:** Habilidad para estructurar entornos reproducibles y documentar de manera efectiva pruebas de endpoints y servicios web utilizando herramientas de validación.

2. Resultado del Aprendizaje

2.1. Objetivo SMART

Define el/los objetivo(s) que se pretende cumplir con la práctica planteada, con un enfoque SMART, tomando en consideración que un objetivo SMART va enfocado a la habilidad (y no tanto a la teoría).

Específico (¿Qué?)	**Medible (¿Cuánto?)**	**Alcanzable (¿Cómo?)**	**Realista (¿Para qué?)**	**A Tiempo (¿Cuándo?)**
Diseñar e implementar un backend (API REST) para la base de datos "Comercial La Estrella", delegando reglas de negocio complejas y análisis de datos al motor mediante el uso de Triggers y Vistas.	Entregando la infraestructura dockerizada, endpoints CRUD para las tablas existentes, 3 vistas analíticas, 2 triggers funcionales y una colección de pruebas documentada.	Utilizando Docker para la infraestructura, SQL Server (T-SQL) para la base de datos, Node.js/Express para la API, y Postman para la validación de peticiones.	Para simular un entorno laboral moderno donde se prohíbe la manipulación directa de datos, asegurando la integridad a través de servicios web y el motor de BD.	Dentro del plazo de elaboración establecido en el cronograma oficial del proyecto.

3. Enunciado del Proyecto

Esta sección define de manera clara y detallada los aspectos específicos que los estudiantes deberán abordar en la práctica. Se incluye el problema a resolver, el alcance, y los entregables esperados.

3.1 Descripción del problema a resolver

La gerencia de *Comercial La Estrella* ha aprobado con éxito el modelo relacional diseñado en la Fase 1 para el control de sus tiendas, empleados, clientes, productos, ventas y pagos. Ahora, el departamento de TI requiere transformar ese modelo teórico en una solución de backend funcional, estandarizada y segura.

Para lograrlo, han decidido migrar el motor de base de datos a **Microsoft SQL Server 2022** y utilizar **Docker** como estándar de infraestructura. El Administrador de Base de Datos (DBA) ha establecido una política estricta de seguridad: **ningún usuario ni sistema externo puede manipular datos conectándose directamente a la base de datos.** Todo ingreso, actualización, eliminación o consulta de información debe pasar exclusivamente a través de un **API REST** desarrollado en Node.js.

Adicionalmente, el DBA ha determinado que el servidor de Node.js no debe sobrecargarse realizando cálculos analíticos pesados ni validaciones críticas de integridad, ya que esto le corresponde al motor de base de datos. Por ello, se le solicita implementar los siguientes objetos en SQL Server mediante Transact-SQL (T-SQL):

Reglas de Negocio a nivel de Motor (Triggers):

- **Protección de Inventario (Trigger Insert_Detalle):** Al intentar registrar un producto en una venta a través de la API, el disparador debe verificar si la cantidad solicitada es mayor a la existencia_actual del producto. Si no hay stock suficiente, el disparador debe rechazar la transacción (RAISEERROR / ROLLBACK). Si la transacción es válida, el disparador debe descontar automáticamente la cantidad vendida del inventario.
- **Estado Automático de Venta (Trigger Insert_Pago):** Al registrar un pago asociado a una venta, el disparador debe sumar todos los pagos de dicha venta. Si la suma alcanza o supera el monto total de la venta, el motor debe actualizar automáticamente el estado de la venta a "PAGADA".

Consultas Analíticas (Vistas):

El API REST no debe realizar JOINs complejos en su código fuente. Debe crear las siguientes Vistas en SQL Server y hacer que los endpoints de reportes simplemente las consulten (ej. SELECT * FROM Vista_Ventas_Ubicacion;):

1. **Vista_Ventas_Ubicacion:** Muestra la tienda, municipio, departamento, país, cantidad de ventas y total facturado (excluyendo ventas anuladas).
2. **Vista_Top_Productos:** Muestra el código del producto, nombre, categoría, marca, unidades totales vendidas y monto total generado (excluyendo ventas anuladas).
3. **Vista_Saldos_Pendientes:** Muestra únicamente las ventas con estado "Registrada", indicando el total de la venta, el monto total pagado (si ya existe algún abono) y la diferencia (saldo pendiente).

3.2 Alcance del Proyecto

El proyecto comprende el desarrollo completo de un backend funcional, dividiendo las responsabilidades entre la base de datos (Docker + SQL Server) y la capa de aplicación (Node.js). El estudiante deberá ser capaz de implementar:

A) Implementación del API REST (Operaciones CRUD)
El estudiante debe programar un servidor web (utilizando Express en Node.js) que exponga los endpoints HTTP necesarios para gestionar las entidades principales del negocio (Clientes, Productos, Tiendas, Empleados, Ventas, Pagos y Detalles de Venta). El API debe ser capaz de procesar y responder en formato JSON para las siguientes operaciones:

- **Creación (POST):** Endpoints para registrar nueva información en las tablas.
- **Lectura (GET):** Endpoints para consultar listados completos o buscar registros individuales por su ID.

- **Actualización (PUT):** Endpoints para modificar la información de registros existentes.

- **Eliminación (DELETE):** Endpoints para borrar registros (o cambiar su estado lógico, según lo requiera el modelo).

B) Casos de Validación (Pruebas de Triggers mediante Postman) El estudiante debe garantizar que la conexión entre su API y SQL Server capture y maneje correctamente los errores emitidos por el motor. Durante la revisión, se exigirá que el sistema responda de la siguiente manera a estos escenarios de prueba:

1. **Escenario de Inventario (Fallo):** Si se envía un POST para registrar un producto en una venta pidiendo una cantidad mayor a la existencia_actual, la base de datos debe abortar la transacción (Trigger). El API en Node.js debe capturar este evento y devolver a Postman un error (ej. HTTP 400 Bad Request) con el mensaje "Stock insuficiente".

2. **Escenario de Inventario (Éxito):** Si se envía un POST con una cantidad válida, la petición debe devolver éxito (HTTP 200/201). Al ejecutar inmediatamente un GET sobre ese producto, su existencia debe verse reducida automáticamente.

3. **Escenario de Estado de Venta:** Al enviar múltiples peticiones POST para registrar abonos parciales a una venta, el API solo debe responder que el pago fue exitoso. Cuando se inserte el pago que cubra la totalidad de la venta, al ejecutar un GET para consultar esa venta, su estado debe mostrar "PAGADA" (efectuado automáticamente por el motor de base de datos).

C) Infraestructura y Restricciones de Acceso

- **Dockerización:** La instancia de Microsoft SQL Server 2022 debe ejecutarse obligatoriamente desde un contenedor Docker (configurado mediante docker-compose), inicializando su estructura mediante un script DDL automático.

- **Prohibición de Acceso Directo:** El uso de clientes gráficos (DBeaver, Data Studio) queda estrictamente limitado a la verificación estructural (solo lectura). Toda manipulación de datos (CRUD) para la calificación debe realizarse consumiendo el API REST.

3.4 Requerimientos técnicos

Define las tecnologías, herramientas o lenguajes de programación que los estudiantes deberán utilizar o integrar en el desarrollo de la práctica.

- **Infraestructura:** Uso de Docker y docker-compose para levantar la base de datos SQL Server.
- **DDL para inicializar la base de datos automáticamente.**
- **Lenguaje de BD:** Sentencias DDL, DML y Transact-SQL (T-SQL) para los objetos programables.
- **Backend:** Entorno de ejecución Node.js utilizando el framework Express (o similar) y librerías de conexión a SQL Server (ej. mssql).
- **Herramientas de testing y administración:** Postman (para consumir la API) y Cliente DBeaver para la administración y verificación de la estructura de BD.
- **CRUD** implementado para todas las tablas.

4. Entregables

Describe los productos concretos que se espera que los estudiantes entreguen al finalizar la práctica. Pueden ser prototipos, informes técnicos, documentación, etc.

Tipo	Descripción
Infraestructura (Docker y DDL)	Archivo docker-compose.yml configurado para SQL Server 2022 y el script de inicialización init.sql (que debe contener la creación de tablas, llaves, vistas analíticas y triggers). Deben estar en la raíz del proyecto.
Código del API REST (Backend)	Código fuente completo del servidor desarrollado en Node.js (incluyendo package.json, configuración de conexión, rutas y controladores por entidad).
Pruebas de Endpoints	Archivo .json exportado desde Postman con la colección completa de peticiones. Debe incluir las operaciones CRUD y las pruebas específicas que demuestran el funcionamiento de los triggers (errores de inventario y cambio de estado).
Documentación Técnica	Archivo README.md o un documento PDF que incluya: instrucciones paso a paso para levantar el contenedor, guía para ejecutar el API REST y capturas de pantalla que evidencien el funcionamiento de los triggers.

- Todos los archivos deben ir dentro de una carpeta comprimida con el formato: **[BD1]_ProyectoFase2_#carnet.zip.** Ejemplo: [BD1]_ProyectoFase2_201709196.zip

5. Material de apoyo

- Modelo Relacional aprobado y Diccionario de Datos de la Fase 1 del proyecto (Comercial La Estrella).
- Documentación oficial de Microsoft SQL Server (Transact-SQL) enfocada en la sintaxis para la creación de Vistas y Triggers.
- Documentación oficial de Docker y sintaxis básica para archivos docker-compose.yml.
- Guía institucional de creación de API REST con Node.js y Express (material del curso).
- Guía de uso de Postman para validación de servicios web y estructuración de peticiones HTTP (GET, POST, PUT, DELETE).

6. Recursos y herramientas a utilizar

Listado de materiales que los estudiantes deberán usar o investigar:

- **Software/Hardware:** Docker Desktop (o motor Docker en Linux/Mac), motor de base de datos Microsoft SQL Server 2022 (mediante imagen de contenedor), entorno de ejecución Node.js, Postman (para pruebas), Visual Studio Code (editor recomendado), y DBeaver (únicamente para visualización estructural). Computadora con recursos suficientes para virtualización (se recomiendan al menos 8 GB de RAM).

- **Plataformas:** UEDI (plataforma oficial de entrega) y Google Classroom (plataforma de respaldo).

- **Librerías recomendadas:** Paquete mssql (vía npm) para establecer la conexión asíncrona entre el servidor Node.js y SQL Server.

7. Cronograma

Tipo	Fecha Inicio	Fecha Fin
Asignación del Proyecto	Viernes 18 de septiembre de 2026	Viernes 18 de septiembre de 2026
Elaboración	Viernes 18 de septiembre de 2026	Jueves 15 de octubre de 2026
Entrega Final (UEDI / Classroom)	Jueves 15 de octubre de 2026	Jueves 15 de octubre de 2026
Calificación	Sábado 17 de octubre de 2026	Sábado 17 de octubre de 2026

8. Rúbrica de Calificación

8.1 Requisitos para optar a la calificación

Antes de la evaluación de la práctica, los estudiantes deben cumplir con los requisitos que se indiquen en esta sección.

Tema	Descripción	Cumple (Sí/No)
Infraestructura Dockerizada	La base de datos Microsoft SQL Server 2022 debe ejecutarse obligatoriamente mediante contenedores utilizando Docker y docker-compose. No se aceptarán instancias locales tradicionales de SQL Server.	
Restricción de Manipulación de Datos	Todas las operaciones de inserción, actualización y eliminación (CRUD) deben realizarse exclusivamente a través del API REST. El uso de DBeaver o herramientas gráficas queda restringido únicamente a la verificación estructural (solo lectura).	
Gestión y Entregas	Entregar la carpeta comprimida con el formato [BD1]_ProyectoFase2_#carnet.zip con todos los archivos requeridos.	
Evidencia de Pruebas	Adjuntar la colección exportada de Postman con las peticiones funcionales y los escenarios de validación que demuestren el funcionamiento de los Triggers y Vistas.	

La evaluación del proyecto se realizará en función de varios criterios clave, teniendo en cuenta tanto los aspectos técnicos como las habilidades blandas demostradas a lo largo del desarrollo.

8.2 Resumen de Puntuaciones

Área	Puntos Totales	Puntos Obtenidos
1. Habilidades		
API (CREATE, READ, UPDATE, DELETE, Carga de Datos)	16	
Dockerización y Conexión	16	
Documentación	15	
Sub-Total Habilidades	**47**	
2. Conocimiento		
Vista 1	5	
Vista 2	5	
Vista 3	5	
Trigger 1	5	
Trigger 2	5	
Pregunta teórica	8	
Sub-Total Conocimiento	**33**	
Ejercicio práctico	**20**	
TOTAL	**100**	

Detalle de la Calificación

1. Habilidades (40 pts)

No.	Criterio de evaluación	Punteo máximo	Satisfactorio 100% - 61%	Necesita mejorar 60% - 0%	Punteo Obtenido
1.1	API (CREATE, READ, UPDATE, DELETE, Carga de Datos)	16	La API en Node.js expone correctamente los endpoints para las operaciones CRUD. Procesa peticiones HTTP, responde con JSON adecuado y permite la carga de datos masiva o inserciones sin fallos lógicos.	Faltan operaciones CRUD críticas, la API crashea al recibir datos incorrectos, o no se maneja adecuadamente la inserción de datos en la base de datos.	
1.2	Dockerización y Conexión	16	El entorno se levanta exitosamente con docker-compose. SQL Server se inicializa automáticamente ejecutando el script DDL, y la API se conecta al contenedor de manera fluida mediante variables de entorno.	El contenedor falla al iniciar, el script de SQL no se ejecuta automáticamente, o la API no logra conectarse por una mala configuración de infraestructura.	
1.3	Documentación Analítica	15	Se entrega el manual técnico/README con instrucciones precisas de ejecución, el enlace al repositorio de GitHub y la colección de pruebas de Postman completamente funcional y bien estructurada.	Las instrucciones de despliegue son nulas o ambiguas, el repositorio carece de accesos, o la colección de Postman está incompleta/ausente.	

2. Conocimiento (60 pts)

No.	Criterio de evaluación	Punteo máximo	Satisfactorio 100% - 61%	Necesita mejorar 60% - 0%	Punteo Obtenido
2.1	Explicación de Vista 1, 2 y 3	15 (5 c/u)	El estudiante demuestra y explica de manera fluida la lógica T-SQL implementada en las 3 vistas. Los resultados en Postman reflejan los JOINs y cálculos requeridos de forma precisa.	Las vistas presentan errores en los cálculos o uniones, no devuelven la información solicitada, o el estudiante desconoce cómo funcionan internamente.	
2.2	Explicación de Trigger 1 y 2	10 (5 c/u)	Los disparadores de inventario y estado funcionan correctamente en vivo. El estudiante explica el uso de las tablas temporales (INSERTED) y el control de transacciones en el motor.	Los triggers no logran evitar validaciones erróneas, no actualizan los estados esperados, o el estudiante no comprende su funcionamiento transaccional.	
2.3	Pregunta teórica	8	El estudiante responde correctamente, de forma técnica y segura, a la pregunta formulada por el tutor sobre conceptos de SQL Server, APIs o infraestructura Docker.	El estudiante no logra formular una respuesta coherente con la pregunta teórica o denota falta de dominio conceptual.	
2.4	Ejercicio práctico	20	El estudiante comprende el nuevo requerimiento, analiza su código y resuelve de forma lógica y en el tiempo estipulado la modificación solicitada en la BD o en el endpoint de la API.	El estudiante excede el tiempo estipulado, no logra estructurar la solución o demuestra desconocimiento de la arquitectura de su propio proyecto.	